

CUDA Threading Concepts

Parallel and Heterogeneous Computing

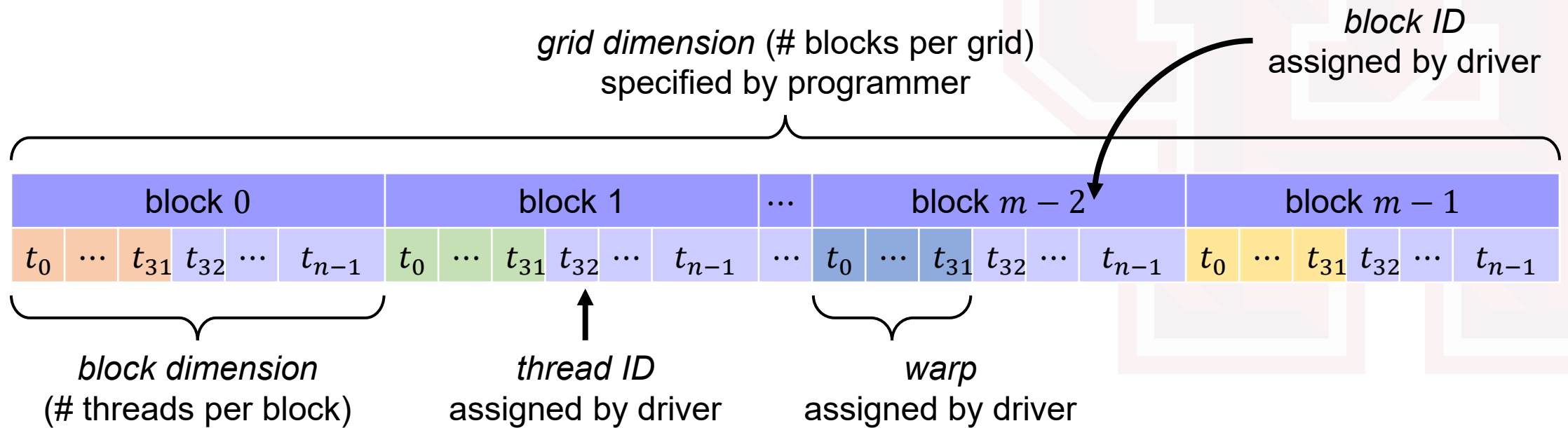
David Mayerich

Scalable Tissue Imaging and Modeling (STIM) Laboratory
Department of Electrical and Computer Engineering
Cullen College of Engineering
University of Houston

UNIVERSITY of **HOUSTON** | ECE

CUDA Thread Organization

- Programmer arranges threads into a **grid**
- A grid is composed of **blocks** of threads
- The processor executes threads in SIMD **warps**
- All threads in a block are executed on a single **streaming multiprocessor**



Launching Threads

- User specifies parameters of the grid layout
- Example:
 - a grid dimension of 100 blocks
 - a block dimension of 256 threads
 - GPU launches $100 \times 256 = 25600$ threads
- All threads execute the same kernel

```
__global__ void kernelFunc(...) {  
    // kernel code executed on GPU  
}  
// call kernel configuration parameters  
kernelFunc <<<blocks, threads>>>(...)  
  
kernelFunc <<<100, 256>>>(...)
```



Declaring and Launching CUDA Kernels

```
//define a kernel
__global__
void kernelAbs(float* in, float* out, int N){
    ...
    ...
}

//launch the kernel in a host function
void main(int argc, char* argv[]){
    ...           //initialize function parameters
    gridDim = ...; //initialize configuration parameters
    blockDim = ...;

    //launch kernel
    kernelAbs<<<gridDim, blockDim>>>(gpu_in, gpu_out, N);
}
```

Data Parallelism in Kernels

- If all threads execute the same kernel, how do you operate on different data?
- Each thread has an independent set of parameters available assigned by CUDA at launch

`gridDim`

specifies the number of blocks in the grid

`blockDim`

specifies the number of threads in a block

`blockIdx`

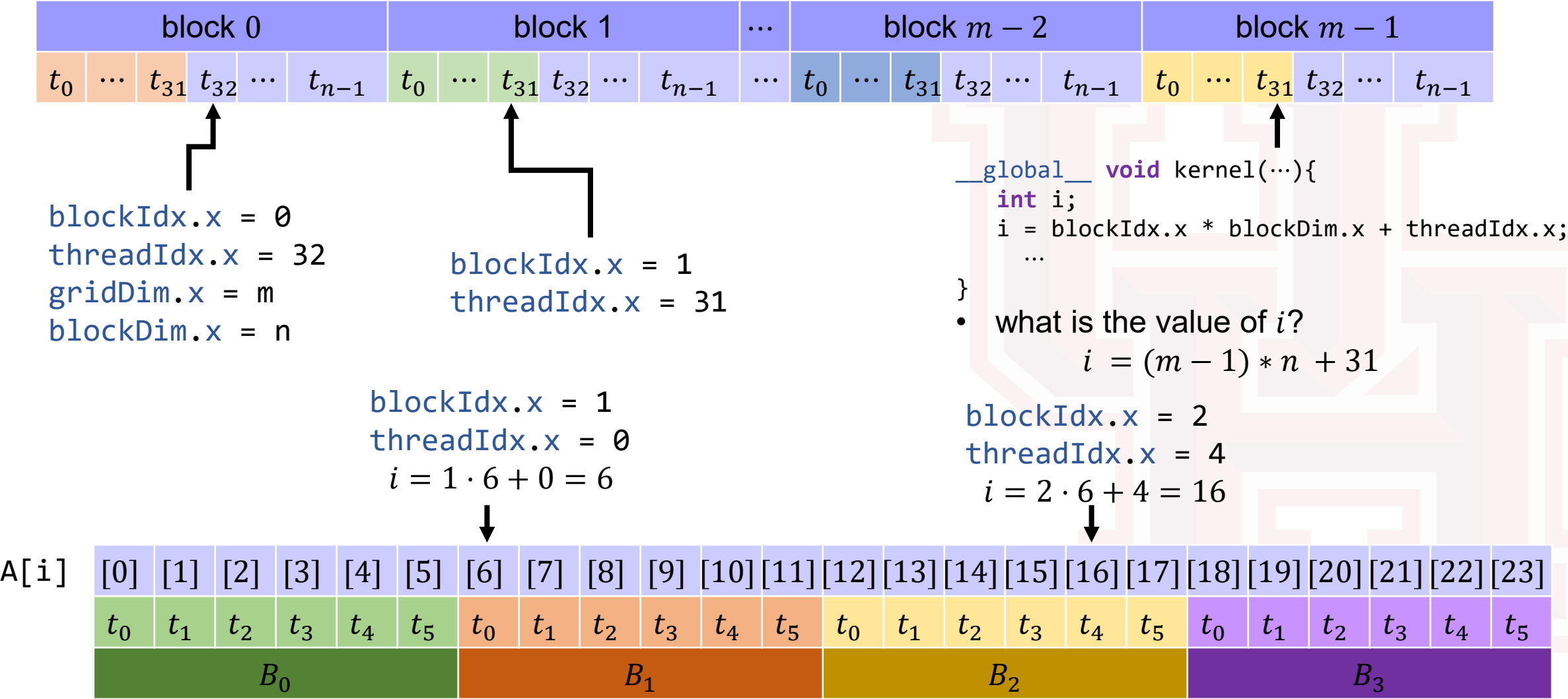
gives a unique identifier for the current block

`threadIdx`

gives a unique identifier for the current thread

- Each parameter is a *triple*: (x, y, z)
 - grids and blocks can be specified in three dimensions
 - `gridDim.x` gives the number of blocks along x
 - `threadIdx.y` gives the identifier along the y dimension within this block
- For now, we will stick to 1D grids (only the x dimension is used)

Data Parallelism in Kernels



Absolute Value of an Array (CPU)

calculate $B_i = |A_i|$ where $i \in [0, N - 1]$

```
//declare and define the abs( ) function
float abs(float x){
    if(x < 0) return -x;
    return x;
}
```

```
int main(int argc, char* argv[]){
    //N is the number of values in the array

    //calculate the number of bytes in the array
    size_t bytes = sizeof(float) * N;
    //allocate space for A
    float* A = malloc(bytes);
    //load the array from a file
    //allocate space for B
    float* B = malloc(bytes);
    //for each value in the array
    for(int i = 0; i < N; i++){
        //calculate the absolute value
        B[i] = abs(A[i]);
    }
```

Absolute Value of an Array: __host__

```
int main(int argc, char* argv[]){
    //N is the number of values in the array
    size_t bytes = sizeof(float) * N;
    float* A = malloc(bytes);
    //load the array from a file
    float* B = malloc(bytes);
    float* gpuA;
    //allocate memory on the CUDA device and copy A
    HANDLE_ERROR(cudaMalloc(&gpuA, bytes));
    HANDLE_ERROR(cudaMemcpy(gpuA, A, bytes, cudaMemcpyHostToDevice)); //copy A to gpuA (on device)
    float* gpuB;
    HANDLE_ERROR(cudaMalloc(&gpuB, bytes));
    HANDLE_ERROR(kernelAbs<<<N, 1>>>(gpuB, gpuA, N));
    HANDLE_ERROR(cudaMemcpy(B, gpuB, bytes, cudaMemcpyDeviceToHost)); //copy result to main memory
    //the result is in B

    //calculate the number of bytes in the array
    //allocate space for A

    //allocate space for B
    //create a pointer (device memory)

    //allocate device memory
    //create a pointer (device memory) to store the result
    //allocate device memory
    //launch the kernel
    //copy result to main memory
```

Absolute Value of an Array: `__global__`

A[i]	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]	...	[N - 3]	[N - 2]	[N - 1]
	B_0	B_1	B_2	B_3	B_4	B_5	B_6	B_7	B_8	B_9	B_{10}	B_{11}	...	B_{N-3}	B_{N-2}	B_{N-1}

```
__global__ void kernelAbs(float* out, float* in, int N){
    int i = blockIdx.x;
    if(in[i] < 0) out[i] = -in[i]
    else out[i] = in[i];
}
```

- What are the configuration parameters for the following grid layout?

[0]	[1]	[2]	[3]	[4]	[5]	...	[N - 6]	[N - 5]	[N - 4]	[N - 3]	[N - 2]	[N - 1]
t_0	t_1	t_2	t_3	t_4	t_5	...	t_0	t_1	t_2	t_3	t_4	t_5
B_0						...	B_{M-1}					

```
kernelAbs<<< M, 6 >>>(gpuB, gpuA, N);
```

- How about the kernel index i ?

```
i = blockIdx.x * blockDim.x + threadIdx.x;
```

- There are a limited number of threads per block
 - currently usually around 1024
- The grid and block dimensions are all limited
 - max size of any block dimension is usually the max number of threads (≈ 1024)
 - max size of any grid dimension is usually ≈ 65535
- All parameters can be queried with `cudaError_t`
`cudaGetDeviceProperties(
 cudaDeviceProp* prop, int dev)`

```
struct cudaDeviceProp{  
    char name[256];  
    size_t totalGlobalMem;  
    int maxThreadsPerBlock;  
    int maxThreadsDim[3];  
    int maxGridSize[3];  
    ...  
    int major;  
    int minor;  
    ...  
    //more GPU features  
}
```

Warps

- GPU block architecture is broadly a SPMD:
 - single program multiple data (SPMD) – programming architecture where a single program is executed on data
 - threads generally don't have to execute the same instruction at the same time
- The GPU organizes threads within a block into *warps*
- *warp* – a series of threads in a block that execute the same instruction in a SIMD manner
- The warp size is generally ≈ 32 threads
- Help mitigate the problem of thread divergence
 - divergent threads (loops, conditionals) only stall the warp
- Scattered divergent threads can still cause damage in the form of stalls!

How CUDA Grids Map to Architecture

- All threads in a block will run on the same streaming multiprocessor (SM)
 - an SM can execute multiple threads
 - not all threads will start or finish at the same time
 - all threads in a block have access to the same *shared memory*
- Maximizing throughput $\approx$ maximizing occupancy
 - maximize the number of threads running simultaneously on an SM
 - limiting factors include:
 - number of registers used (variables in the kernel)
 - shared memory used (discussed later)
 - stalls (thread divergence)

Techniques for Designing Grid Layouts

- If you expect low thread divergence, make blocks a multiple of the warp size (usually 32)
- Minimizing the number of registers used in the kernel will allow more blocks to be executed
- Map the grid onto your data
 - 1D arrays should use a 1D grid (`threadIdx.x`, `blockDim.x`, etc.)
 - 2D problems often use a 2D grid (x, y)
- **image processing**: map one thread to each output pixel
- **matrix multiplication**: map one thread to each value M_{ij} in the output matrix